Lazy orbit-member index in the Frobenius-folded pair table

2026-10-18. Study `research/ecbench_lazy_orbit_members_20261018/`, protocol `PROTOCOL.md` (frozen in commit `0334afb11` before any claim data). Class: **engineering**. Sessions ran on that commit (`git_dirty: false`), `ecbench` binary `2689809e…`, host class `ECBENV2he1389885ccd0` (4-vCPU Xeon VM, no CPU reservation: isolation L0, so operation counts stand and wall times are descriptive only).

Result

The frozen orbit index calculus on Koblitz curves spent a large part of its set-up naming, for every base point P_i and every rotation t, the base index of $\sigma^{t(P_i)}$: $|F|(n-1)$ Frobenius maps and as many hash lookups. Only a probe that hits reads two of those entries (about 1–2 % of probes). The new oracle `mitm-frobenius-lazy-counted` builds the same table and probe and names the two summands of a hit when it happens, charging that work to the relation phase. Nothing else differs between the two IC arms.

Measured (counted `ecbench.gae`, every counter priced, paired by target): the lazy arm costs less than the frozen baseline on all 8 curves, with the lower end of the 95 % interval above 1 on every one (table 1, figure 1). The saving is $1.12\times$ to $1.44\times$ on the 7 curves with $\log_2 r \le 28$ and $1.014\times$ at $n = 41$ (c = 90 columns, where the member index is a small part of S).

Same algorithm (H1): on all 856 baseline/lazy run pairs (every target and pass, warm-ups included; 4 280 verified records in all), the trials, relations, matrix rank, recovered logarithm and table entry count are identical between the arms (jq check over `records.jsonl`; 0 mismatches). All records verified, none is a lower bound, every audit passed with all replays reproduced (`audits/`), and `rho-frob-aa ÷ rho-frob` is 1 [1, 1] on every curve.

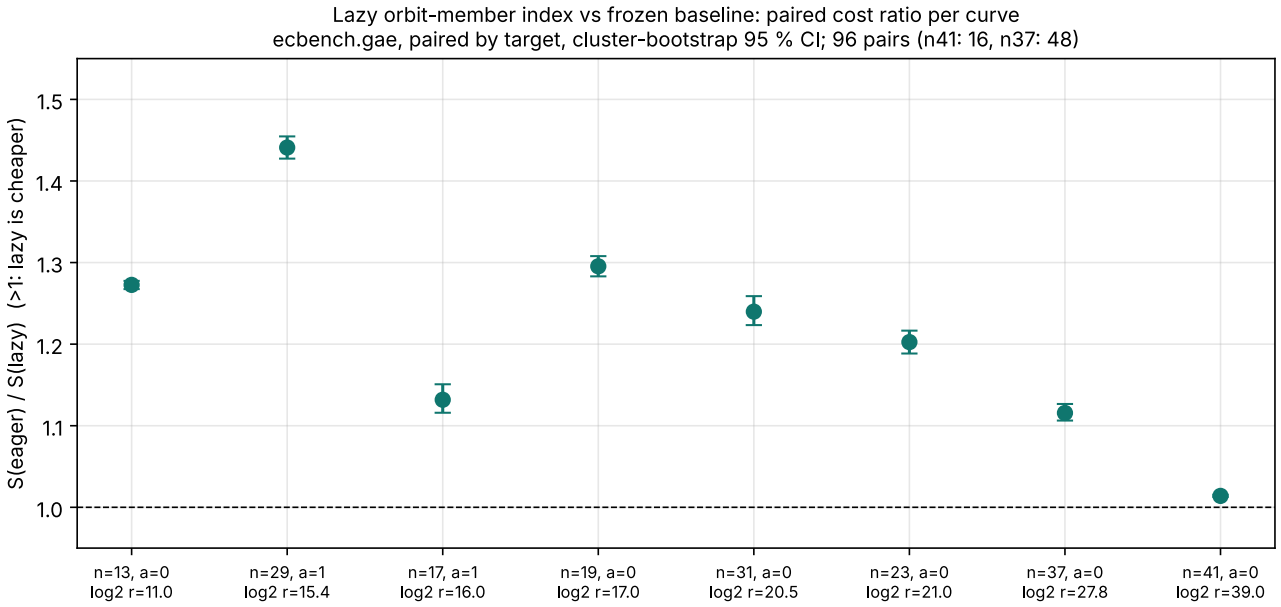


Figure 1: Eager ÷ lazy paired cost ratio per curve. Sessions ECBS1hbd6dafc7363a (n13), ECBS1h2a2479247f55 (n17), ECBS1h5149dde01d5c (n19), ECBS1h7daf91450aa7 (n23), ECBS1h807488a20506 (n29), ECBS1h9649ebbe0eac (n31), ECBS1h74c2d8ce9372 (n37), ECBS1h81fbae75c542 (n41).

curve (a, n)	$\log_2 r$	eager ÷ lazy (H2)	eager ÷ rho-strong (was)	lazy ÷ rho-strong (H3)	lazy ÷ rho-frob
0, 13	11.0	1.273 [1.267, 1.278]	0.409 [0.400, 0.419]	0.322 [0.314, 0.330]	1.912 [1.291, 2.505]
1, 29	15.4	1.441 [1.427, 1.455]	0.577 [0.562, 0.593]	0.401 [0.387, 0.415]	1.272 [0.770, 1.803]
1, 17	16.0	1.132 [1.116, 1.151]	0.719 [0.642, 0.806]	0.635 [0.558, 0.722]	1.748 [1.305, 2.350]
0, 19	17.0	1.295 [1.283, 1.308]	0.745 [0.716, 0.777]	0.575 [0.549, 0.605]	2.011 [1.599, 2.446]
0, 31	20.5	1.240 [1.223, 1.259]	1.221 [1.142, 1.300]	0.985 [0.908, 1.062]	2.646 [1.997, 3.361]
0, 23	21.0	1.203 [1.189, 1.217]	1.342 [1.250, 1.449]	1.116 [1.028, 1.218]	2.881 [2.480, 3.333]
0, 37	27.8	1.116 [1.106, 1.127]	4.260 [3.774, 4.823]	3.818 [3.367, 4.345]	4.746 [3.988, 5.710]
0, 41	39.0	1.014 [1.013, 1.015]	16.09 [11.71, 22.27]	15.87 [11.53, 21.98]	13.98 [10.05, 21.41]

Table 1: Paired ratios, cluster-bootstrap 95 % intervals (`ecbench compare`, `sessions/*/comparisons/`). 96 pairs per curve (32 targets × 3 rounds); n37: 48; n41: 16. Columns 4 to 6 are costs of the first-named arm divided by rho; below 1 the IC arm is cheaper.

Distance to rho (reported, not a gate)

The gap to the matched strong rho narrowed on every curve. It closed on the small curves already (IC below `rho-strong`), is now a tie at $n = 31$ (0.985 [0.908, 1.062], was 1.221 [1.142, 1.300]) and is still above 1 at $n = 23$ (1.116, was 1.342), $n = 37$ (3.82, was 4.26) and $n = 41$ (15.9). Against `rho-frob` the lazy arm's point estimate stays above 1 on every curve; only the $n = 29$ interval (0.77 to 1.80) includes 1. No ranking against rho is claimed: the order is not shown to survive AGENTS.md §5a's pricing band, and `ecbench` wall times at L0 are not admitted. At $n \geq 23$ IC remains far from beating rho and the gap grows with r (figure 2), as the 2026-10-14 protocol found.

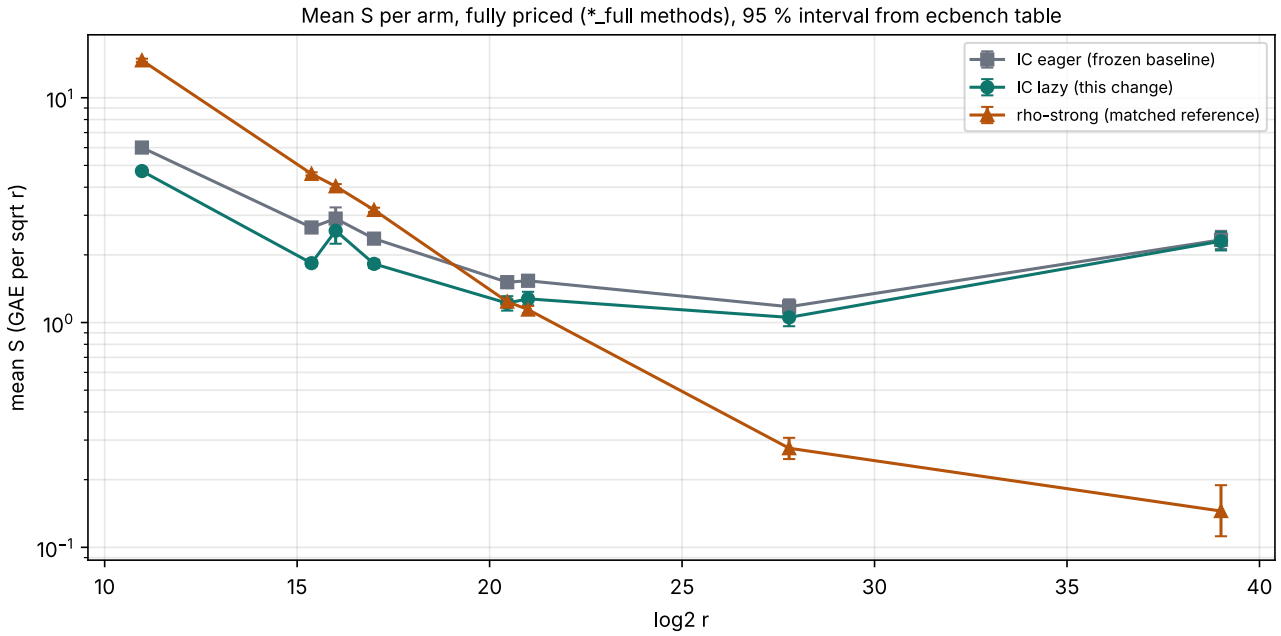


Figure 2: Mean S (GAE per $\sqrt{r}$) per arm against $\log_2 r$, interval from `ecbench table`.

Where the saving is (H4)

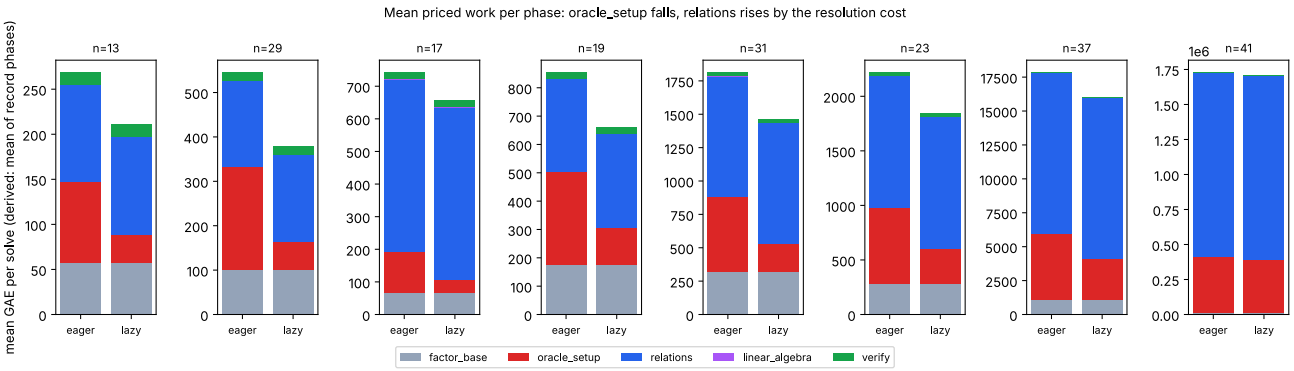


Figure 3: Mean priced work per phase (derived: mean over each arm's non-warm-up records of that phase's `gae`). `oracle_setup` falls; `relations` rises by the resolution cost (3.4 GAE of 1 214 at n23).

At $n = 23$ (c = 3) `oracle_setup` falls from 695 to 318 GAE and `relations` rises from 1 214.3 to 1 217.7 GAE: net -374 GAE of 2 220. The resolution costs two index lookups and $t \leq n - 1$ Frobenius maps per summand per hit, charged at the unit price the eager table paid per step.

Classification and limits

Engineering (AGENTS.md success criterion): dead set-up work removed from one oracle; same relations, same exponent. It is not relabelling (the trial and relation counts are unchanged and the counters moved with the work) and not accounting (the saving is native work no longer performed, charged at the same unit prices). Per-curve columns are those of the 2026-10-14 protocol and were not retuned; exploratory sessions (not kept) suggested fewer columns suit the lazy oracle, a next experiment. Prices are one host's microbenchmark ratios. Nothing here concerns larger fields, ECC2K-130, deployed curves or security.

Evidence: `sessions/` (8 ecbench sessions, loose, CI-re-auditable), `audits/`, `tables/`, `logs/`, `figures/summary.json` (derived means) and `figures/plot_figures.py` (lays out recorded values only). Unit test `lazy_frobenius_table_returns_the_eager_pairs_and_charges_the_resolution`; `cargo test --release --lib -- cryptanalysis::ic_boundary cryptanalysis::ic_framework cryptanalysis::ecbench`: 204 passed.